

Frontend / Design Engineer Take Home Assignment

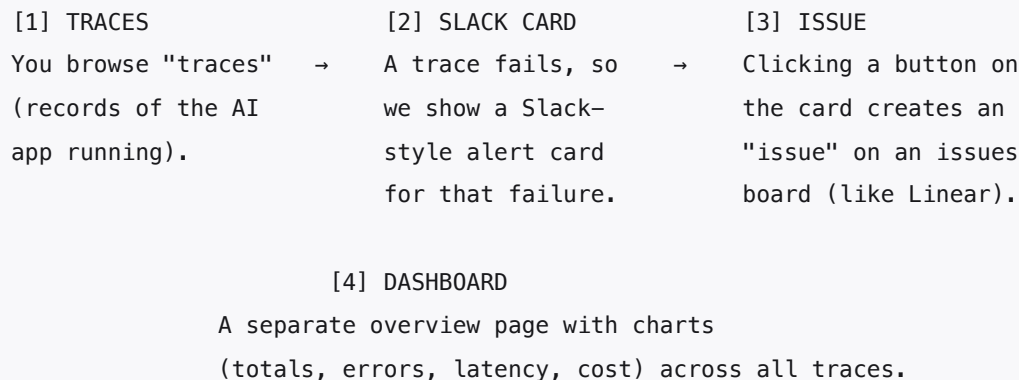
Thanks for taking the time to do this. This document tells you **exactly** what to build. Please read it fully before you start — it's long on purpose, so there's no guesswork.

Remember one thing: **this is a frontend + design exercise**. You're building screens in the browser using fake data we give you. There is **no backend, no database, no login, and no real Slack** anywhere in this task.

1. The big picture (read this first)

You're building a small version of an **LLM observability tool** — the kind of dashboard a team uses to watch their AI app and catch problems.

The whole app is **one connected story**. Follow this flow:



In simple terms:

1. There's a page that **lists traces** (think LangSmith).
2. Some traces have **failed**. For a failed trace, you show a **card that looks like a Slack message** (an alert).
3. That card has a **"Create issue" button**. Clicking it makes a new **issue** and puts it on an **issues board that looks like Linear**.
4. There's also a **dashboard** page with charts summarising everything.

That's the whole app. Four screens, connected.

2. Very important: what this is NOT

- You are **NOT** building a real Slack app, Slack bot, or Slack integration.

- You are **NOT** calling the Slack API or any other API.
- You are **NOT** building a backend, server, or database.
- You are **NOT** building login / auth.
- The "Slack card" is just **UI in the browser that looks like a Slack message**, built from a JSON file we give you.
- "Creating an issue" just means **adding an item to a list in the browser** (React state / a store). It does not save anywhere.
- Everything reads from the **two JSON files** we provide.

So nothing here is full-stack. It's all frontend.

3. The data we give you

Two files. You can load them however you like.

`traces.json` — the list of AI app runs (~50 of them)

A **trace** = one run of the AI app. A trace contains **spans** = the individual steps inside that run (calling the AI model, calling a tool, etc.). Spans form a tree (a span can have child spans).

A trace looks like this (trimmed):

```
{
  "id": "trace_g0116",
  "name": "customer_support_agent",
  "status": "error",          // "success" | "error" | "running"
  "startTime": "2026-06-13T22:43:22.134Z",
  "endTime": "2026-06-13T22:43:24.436Z",
  "latencyMs": 2302,          // how long it took
  "totalTokens": 3019,        // total tokens used
  "totalCostUsd": 0.008,      // total cost
  "tags": ["production", "support"],
  "metadata": { "environment": "production", "userId": "user_8842" },
  "feedback": { "rating": "up", "score": 0.41, "comment": null }, // optional
  "spans": [
    {
      "id": "trace_g0116_s0",
      "parentId": null,        // null = top of the tree
      "name": "AgentExecutor",
      "type": "chain",          // "chain" | "llm" | "tool" | "retriever" | "parser"
      "status": "error",
      "startTime": "...",
      "endTime": "...",
    }
  ]
}
```

```

    "latencyMs": 2302,
    "input": { "...": "..." }, // any JSON
    "output": null,
    "error": "ToolException: connection refused" // only when it failed
  },
  {
    "id": "trace_g0116_s1",
    "parentId": "trace_g0116_s0",
    "name": "ChatOpenAI",
    "type": "llm",
    "status": "success",
    "model": "gpt-4o",           // these 5 fields exist ONLY on "llm" spans:
    "promptTokens": 1180,
    "completionTokens": 64,
    "totalTokens": 1244,
    "costUsd": 0.008,
    "input": { "...": "..." },
    "output": { "...": "..." }
  }
]
}

```

Things to know:

- **status** can be **success**, **error**, or **running**.
- A **running** trace/span is still going: it has **no endTime** and **no latencyMs**, and its **output** is **null**. Don't crash on these.
- Token/cost fields on a **span** appear **only on llm spans**. The **trace** has its own **totalTokens** / **totalCostUsd** summary — you can trust those or add up the spans yourself, just be consistent.
- **feedback** exists on some traces only.

slack-cards.json — example Slack messages (6 of them)

This file shows you what the Slack cards should look like. Each message is built from **blocks** — Slack's format for messages. Your job is to turn these blocks into something that **looks like a real Slack message**.

```

{
  "id": "msg_alert_01",
  "traceId": "trace_g0116",           // which trace this card is about
  "lifecycle": "alert",
  "blocks": [
    { "type": "header", "text": { "type": "plain_text", "text": ":rotating_light: Trace failed" } },
    { "type": "section", "text": { "type": "mrkdwn", "text": "A *production* run hit a `connect" } },
    { "type": "section", "fields": [
      { "type": "mrkdwn", "text": "*Severity:*\\nHigh" },
    ] }
  ]
}

```

```

    { "type": "mrkdwn", "text": "*Model:*\\ngpt-4o" }
  ]},
  { "type": "divider" },
  { "type": "actions", "elements": [
    { "type": "button", "text": { "type": "plain_text", "text": "View Trace" }, "url": "...
    { "type": "button", "text": { "type": "plain_text", "text": "Create issue" }, "style":
  ]}
]
}

```

The block types you'll see: `header`, `context`, `section` (with `text` and/or a two-column `fields` array), `divider`, and `actions` (which holds `button` s and a `static_select` dropdown).

The text uses Slack's **mrkdwn** format. You need to render a sensible subset:

- `*bold*` → **bold**
- `_italic_` → *italic*
- ``code`` → inline code
- ````code fences```` → code block
- `<https://url|label>` → a link
- `:emoji:` → an emoji (e.g. `:rotating_light:` 🚧)
- `\n` → a line break

A note on the data

It's modeled on real production data, so it isn't perfectly clean — expect the odd missing or unexpected value. Handle it gracefully and make sensible calls, and mention anything you noticed in your README.

4. The screens to build

Build these four screens. Suggested routes are in brackets — use them or your own.

Screen 1 — Traces list + detail `[/traces]` and `[/traces/:id]`

Make this look and feel like LangSmith (a clean, dense, functional data tool).

The list (`/traces`):

- A table/list of all traces. Each row shows: **name, status, start time, latency, tokens, cost**.
- Dense and easy to scan. Use monospace for IDs and line numbers up neatly.
- **Success, error, and running** traces must look clearly different at a glance.
- Add **search by name** and **filter by status**.

Done when: I can open `/traces`, scan all ~50 traces, instantly spot the failed ones, search by name, and filter to just errors — without it feeling cluttered or slow.

The detail (`/traces/:id`):

- Show the trace's **span tree** (the steps, nested by `parentId`).
- Click a span → see its details: `input` , `output` (shown as readable JSON), timing, type, and for `llm` spans the model + tokens + cost.
- Show a summary at the top: total time, total tokens, total cost.
- Handle `error` and `running` spans as proper designed states (not blank, not broken).

Done when: I can open any trace, see its steps as a tree, click a step and read what went in and out, and failed/running spans look intentional.

Screen 2 — The Slack alert card (lives on a failed trace)

When I'm looking at a **failed trace**, show the **Slack-style alert card** for it (match it by `traceId` from `slack-cards.json` ; for failed traces that don't have a provided card, you may build a simple alert card from the trace's own data).

- The card must **look like a real Slack message** — header, text with formatting, two-column fields, divider, and styled buttons (note the green **primary** and red **danger** button styles).
- One of the buttons is **"Create issue"**. Clicking it → does Screen 3's job (creates an issue). Other buttons (like "View Trace") can just navigate.

Done when: on a failed trace I see a card that genuinely looks like Slack (formatting and buttons correct), and clicking "Create issue" creates an issue.

Screen 3 — Issues board `[/issues]` and `[/issues/:id]`

Make this look and feel like Linear (the issue tracker). This is the screen we care about most for design.

- When "Create issue" is clicked on a card, a **new issue appears here**, starting with status **Open** by default.
- You decide the issue's fields, but at minimum: an **ID** (e.g. `ISS-001`), a **title** (from the trace/error), a **status**, a **link back to the source trace**, and a **created time**. Linear-style boards also show **priority** and an **assignee** — add what makes the board feel real.
- The board should be **grouped by status** (e.g. Open / In Progress / Resolved), with calm status colors, priority signals, and monospace IDs. You should be able to **change an issue's status** (e.g. a dropdown or drag) — all in the browser.
- An issue's **detail view** (`/issues/:id`) shows its info and **links back to the trace** that caused it.

Done when: creating an issue from a card adds it to the Open group here; I can change its status; and from an issue I can jump to the trace that caused it.

Screen 4 — Dashboard `[/dashboard]`

A focused **overview with charts** across all traces. Keep it small and **honest** — clear charts beat many charts. Include:

- **Stat tiles:** total traces, total cost, total tokens, error rate, and p50 / p95 latency.

- **A latency distribution** (a histogram) with **p50** and **p95** marked.
- **Cost by model** (the data has several models).
- **At least one chart over time** (the data spans ~7 days) — e.g. traces per day, errors per day, or cost per day.

Use any chart library (Recharts, etc.) or plain SVG. Every number must be **calculated from the data** (don't type in fixed numbers). Charts need clear labels/units and a sensible empty state.

Done when: `/dashboard` gives a believable, readable summary, and p50/p95 are real percentiles (not just an average relabelled).

5. How it all connects (the flow)

This is what makes it feel like one product instead of four separate demos:

A trace fails → its Slack alert card appears → I click "Create issue" → a new Open issue shows up on the Linear-style board, linked back to that trace.

Make sure I can walk that whole path in the live app.

6. The look and feel (design matters most)

There's **no mockup** — designing it is your job, and it's the biggest thing we're judging. Match the right reference for each screen:

- **Traces** → **LangSmith**: dense, functional, calm data tool.
- **Issues board** → **Linear**: clean, opinionated, keyboard-friendly, restrained. This is your design showcase.
- **Slack card** → **real Slack**: make it convincingly look like Slack.

General bar: deliberate typography and spacing, color used to mean something (not decoration), clear hierarchy, and smooth, tasteful interactions. Designed empty / loading / error states. Should not break on a smaller screen (we won't nitpick mobile).

7. Tech stack (required)

- **Next.js** (App Router) + **TypeScript**
- **Tailwind CSS**
- **shadcn/ui** for base components

Use any other libraries you'd normally reach for (charts, state, etc.) — just say why in your README.

8. A good order to build it

If you're not sure where to start:

1. Set up the project; load `traces.json`; write clean TypeScript types for a trace and a span.
2. Build the **traces list** (Screen 1) — get it looking like LangSmith.
3. Build the **trace detail** with the span tree.
4. Build the **Slack card renderer** (Screen 2) using `slack-cards.json`.
5. Wire the **"Create issue"** button → build the **issues board** (Screen 3, the Linear one). Spend your best design effort here.
6. Build the **dashboard** (Screen 4).
7. Polish: empty/loading/error states, the messy-data handling, responsiveness.

If you run low on time, **do fewer screens really well** rather than all four half-done — and tell us what you cut.

9. What we're looking for (how we score)

In priority order:

1. **Design & feel** — does it look like a real, polished product (LangSmith for traces, Linear for issues)?
2. **Clean, modular code** — especially the Slack card renderer; reusable pieces, easy to extend, clear types (no `any` everywhere).
3. **The flow works** — trace → card → issue → board all connect.
4. **Slack card accuracy** — formatting, buttons, layout look like real Slack.
5. **Dashboard** — clear, honest charts.
6. **Robustness** — never crashes on the messy/failed/running data.
7. **Communication** — your README explains your thinking.

10. What you do NOT need to do

No backend, no database, no auth, no real Slack, no automated tests, no pixel-perfect mobile. A small, polished, well-built result beats a big messy one.

11. How to submit

Send us **both**:

1. **A live deployed link** — host it anywhere (Vercel is free and perfect; Netlify / Cloudflare Pages are fine too). We want to click into the real thing.
2. **A public GitHub repo** with your full commit history (please don't squash into one commit — we like to see how you work).

In the repo's README, briefly cover:

- How to run it (`install` + `dev` commands).
- **Your design thinking** — what look/feel you went for and why (this matters as much as the code).
- Any trade-offs you made, anything you cut, and anything you noticed in the data.

Time: aim for about **two focused days**. Please don't grind longer.

If anything is unclear, make a reasonable decision and note it in your README — that judgement is part of what we're looking at. Have fun with it.